

# Principio REP

Reuse/Release Equivalence Principle

APLICACIÓN EN TECHFIX LOGISTICS - SISTEMA DE GESTIÓN DE REPARACIÓN DE DISPOSITIVOS

Henry Morales

DEFINICIÓN FORMAL

# Las clases que se reutilizan juntas deben liberarse juntas.

Mismo paquete. Mismo módulo. Misma versión. Unidad cohesiva.

### Solución de Diseño

- **Evita acoplamiento** innecesario.
- **Protege núcleo** de cambios externos.
- **Facilita reutilización** sin dependencias.

# Contexto Histórico

## ORIGEN Y RELACIÓN

### Robert C. Martin

**Autor ("Uncle Bob")**

Década de 1990

### Clean Architecture

**Publicación (2017)**

Formalización de patrones

### Complemento SOLID

**Diseño a Nivel Módulo**

SOLID opera a nivel clase

# El Problema Estructural

## AUSENCIA DEL PRINCIPIO REP

### Acoplamiento Innecesario

Cambios periféricos (ej: formato) afectan núcleo crítico.

Riesgo alto de bugs inducidos.

### Reutilización Imposible

Dependencias opcionales se vuelven obligatorias.

Módulos gigantes inmanejables.

### Versiones Congeladas

Imposible evolución independiente.

Un solo versionado global rígido.

# Implementación Práctica

## PASOS DE APLICACIÓN

1

### Identificar

Analizar frecuencias de cambio.

2

### Separar

Crear módulos cohesivos independientes.

3

### Versionar

Asignar versionado semántico.

4

### Dirigir

Forzar dependencias unidireccionales.

HERRAMIENTAS: Maven • Gradle • JPMS

# Excepciones de Uso

## CUÁNDO EVITAR REP

### Sistemas Monolíticos Simples

Overhead de modularización excede beneficio.

### Prototipos y PoCs

Arquitectura temprana inestable e iterativa.

### Acoplamiento Natural

Componentes del dominio inseparables lógicamente.

### Riesgo de Sobremodularización

Micro-módulos destruyen cohesión interna.

# Monolito TechFix

## ANTES: VIOLACIÓN DE REP

### Estructura Acoplada

- Un solo archivo pom.xml central.
- Servicio central inyecta PriceFormatter.
- Sin control de versiones aislado.

```
// Módulo único sin aislamiento
public class DeviceRepairService {
    private final PriceFormatter priceFormatter;

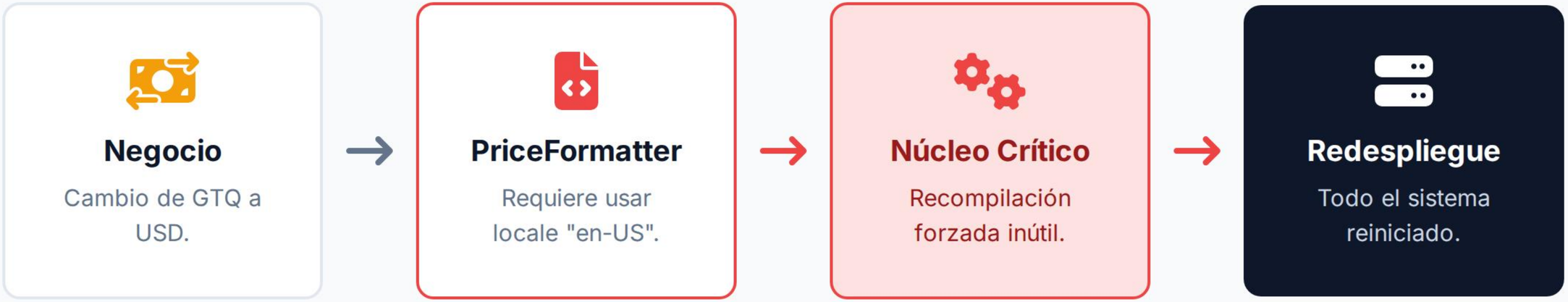
    public DeviceRepairService(
        PriceFormatter priceFormatter) {

        this.priceFormatter = priceFormatter;
    }
}
```

ALTO ACOPLAMIENTO

# Demostración del Fallo

## EL EFECTO CASCADA




**Evidencia en código:** Fallo en `testCouplingViolation_ChangeInFormatterAffectsCoreLogic`.

# Arquitectura Modular

## DESPUÉS: APLICACIÓN DE REP




```java
// repair-core: Sin dependencias de formateo
public class DeviceRepairService {

    public RepairOrder diagnoseAndQuoteRepair(...) {
        BigDecimal cost = calculateRepairCost(...);
        order.setTotalCost(cost);
        return order;
    }
}
```

INDEPENDENCIA TOTAL

# Beneficios Estructurales

## RESULTADOS TRAS APLICAR REP

### Núcleo Blindado

Cero impacto por cambios periféricos.

### Despliegue Aislado

Solo se redespliega el módulo alterado.

### Evolución Libre

Ciclos de versionado semántico propios.

🧪 **Evidencia en código:** `testModuleIndependence_CompilesWithoutFormatterDependency` exitoso.

# Relación con SOLID

## ECOSISTEMA DE DISEÑO ARQUITECTÓNICO

**ISP**

### Segregación de Interfaces

Reducción estricta de dependencias parasitarias.

**SRP**

### Responsabilidad Única

Módulos con clases que cumplen SRP tienen ciclos más claros.

**DIP**

### Inversión de Dependencias

Evolución independiente mediante abstracciones.

**DRY**

### Tensión Controlada

Balance entre repetición e independencia estricta.

# Evidencia Cuantitativa

## MÉTRICAS DE RECOMPILACIÓN

| ESCENARIO DE CAMBIO | IMPACTO "ANTES" (MONOLITO) | IMPACTO "DESPUÉS" (REP) |
|---|---|---|
| Formato de Precios (GTQ a USD) | ⚠ Todo el sistema | ✓ Solo price-formatter |
| Lógica de Cálculo de Costos | ⚠ Todo el sistema | ✓ Solo repair-core |

**100%**

CLASES AFECTADAS (ANTES)

**~50%**

CLASES AFECTADAS (DESPUÉS)

# Resumen Estratégico

CONCLUSIONES Y RECOMENDACIONES

## Beneficios Clave

- Control granular mediante versionado semántico.
- Reutilización limpia sin dependencias ocultas.
- Tiempo mínimo de recompilación y redespliegue.

## Recomendaciones

- [x] Aplicar en sistemas medianos a grandes.
- [x] Balancear fuertemente con la cohesión interna.
- [x] Sincronizar explícitamente con principios SOLID.



# Referencias y Bibliografía


1. **Martin, R. C. (2017).** *Clean Architecture*. Prentice Hall.

2. **Martin, R. C. (2018).** *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.

3. **Martin, R. C. (2002).** *Agile Software Development*. Prentice Hall.

4. **Fowler, M. (2019).** *Refactoring (2nd ed.)*. Addison-Wesley.

5. **Apache Maven / Oracle JPMS.** Documentación técnica oficial.


# ¿Preguntas?